Manual de Migración de Base de Datos

Sistema de Segmentación de Nuevas Especies

SQL Server Azure (valuedata) → SQL Server Azure (adinf)

Empresa:	Garcés Fruit
Proyecto:	Segmentación de Nuevas Especies
Motor origen:	SQL Server Azure
Motor destino:	SQL Server Azure (`adinf.database.windows.net`)
Total tablas:	52 tablas
Estrategia:	Python + pyodbc + T-SQL versionado
Versión doc:	1.0
Fecha:	10 de abril de 2026

Documento confidencial — Uso interno Garcés Fruit

Índice general

1. Introducción **5**
1.1. Propósito del documento . . . 5
1.2. Alcance . . . 5
1.3. Audiencia . . . 6
1.4. Prerrequisitos . . . 6

2. Inventario de Tablas **7**
2.1. Resumen por dominio . . . 7
2.2. Catálogo completo de tablas . . . 7

3. Diagrama de Dependencias **10**
3.1. Grafo de foreign keys . . . 10
3.2. Diagrama ER simplificado . . . 11

4. Arquitectura de la Migración **12**
4.1. Estrategia general . . . 12
4.2. Componentes del sistema de migración . . . 12
4.3. Flujo de ejecución . . . 13

5. Script DDL — Creación de Tablas **14**
5.1. Nivel 0 — Tablas sin dependencias . . . 14
5.2. Nivel 1 — Tablas con dependencias de nivel 0 . . . 21
5.3. Nivel 2 — Tablas con dependencias de nivel 0–1 . . . 25
5.4. Nivel 3 — Tablas con dependencias de nivel 0–2 . . . 28
5.5. Nivel 4 — Tablas con dependencias de nivel 0–3 . . . 32
5.6. Nivel 5 — Tablas finales . . . 36

6. Script Python — Migración de Datos **38**
6.1. Configuración (`config.py`) . . . 38
6.2. Orden de tablas para carga . . . 39
6.3. Script de migración (`migrate.py`) . . . 39
6.4. Script de validación (`validate.py`) . . . 43

7. Procedimiento Paso a Paso **45**
7.1. Fase 1 — Preparación (Día -1) . . . 45
7.2. Fase 2 — Crear estructura (Día 0, hora 0) . . . 46

7.3. Fase 3 — Migrar datos (Día 0, hora 0+15min) 46
7.4. Fase 4 — Validación (Día 0, hora 0+30min) 47
7.5. Fase 5 — Cutover (Día 0, hora 0+45min) 49
7.6. Fase 6 — Rollback (solo si falla) 49

8. Consideraciones Especiales 50
8.1. Columnas VARBINARY (imágenes) 50
8.2. Tabla `usuarios` y hashes bcrypt 50
8.3. Tabla `audit_log` 50
8.4. Collation 51
8.5. Reseed automático de IDENTITY 51

9. Checklist de Migración 52

10. Apéndice A — Variables de Entorno 53

11. Apéndice B — Troubleshooting 54
11.1. Errores comunes 54
11.2. Cómo reintentar una tabla específica 55

12. Apéndice C — Historial de Versiones 56

Índice de cuadros

1.1. Alcance de la migración 5

2.1. Distribución de tablas por dominio 7
2.2. Catálogo completo de tablas del sistema 7

3.1. Orden topológico de creación y carga 10

4.1. Comparación Flyway vs Python 12

9.1. Checklist completo de migración 52

10.1. Variables de entorno requeridas 53

11.1. Errores frecuentes y solución 54

12.1. Historial de versiones del documento 56

Listings

4.1. Estructura de archivos de migración 12
5.1. paises 14
5.2. regiones 14
5.3. campos 15
5.4. especies 15
5.5. portainjertos 15
5.6. pmg 16
5.7. origenes 16
5.8. colores 17
5.9. susceptibilidades 17
5.10. tipos_labor 18
5.11. estados_planta 18
5.12. temporadas 18
5.13. bodegas 19
5.14. catalogos 19
5.15. correlativos 19
5.16. marcos_plantacion 19
5.17. reglas_alerta 20
5.18. roles 20
5.19. defectos 20
5.20. comunas 21
5.21. cuarteles 21
5.22. usuarios 22
5.23. estados_fenologicos 22
5.24. centros_costo 22
5.25. variedades 23
5.26. pmg_especies 24
5.27. portainjerto_especies 24
5.28. viveros 25
5.29. vivero_pmg 25
5.30. variedad_susceptibilidades 25
5.31. testblocks 26
5.32. inventario_vivero 26
5.33. detalles_labor 27
5.34. variedades_log 27
5.35. defectos_variedades 28

5.36. testblock_hileras . . . 28
5.37. posiciones_testblock . . . 29
5.38. plantas . . . 29
5.39. inventario_testblock . . . 30
5.40. guias_despacho . . . 31
5.41. paquete_tecnologico . . . 31
5.42. bitacora_variedades . . . 32
5.43. asignaciones_testblock . . . 32
5.44. movimientos_inventario . . . 33
5.45. historial_posicion . . . 33
5.46. mediciones_laboratorio . . . 33
5.47. ejecucion_labores . . . 35
5.48. registros_fenologicos . . . 35
5.49. alertas . . . 36
5.50. clasificacion_cluster . . . 36
5.51. umbrales_calidad . . . 37
5.52. evidencia_labores . . . 37
5.53. audit_log . . . 37
6.1. config.py — Connection strings . . . 38
6.2. Orden topológico de carga . . . 39
6.3. migrate.py — Orquestador principal . . . 39
6.4. validate.py — Validación post-migración . . . 43
8.1. Reseed automático de todas las tablas . . . 51

Capítulo 1

Introducción

1.1 Propósito del documento

Este manual describe el procedimiento completo, paso a paso, para migrar la base de datos del **Sistema de Segmentación de Nuevas Especies** de Garcés Fruit desde la instancia actual en **SQL Server Azure** a una nueva instancia de **SQL Server** (destino).

El documento cubre:

- Inventario completo de las 52 tablas del sistema
- Scripts DDL para crear la estructura en el destino
- Scripts de extracción y carga de datos (Python + pyodbc)
- Orden de ejecución respetando dependencias de FK
- Procedimiento de validación post-migración
- Plan de rollback ante errores

1.2 Alcance

Cuadro 1.1: Alcance de la migración

Aspecto	Detalle
Origen	SQL Server Azure, servidor `tcp:valuedata.database.windows.net,1433`
Destino	SQL Server Azure, servidor `tcp:adinf.database.windows.net,1433`, BD `adinf`
Esquema	52 tablas distribuidas en 6 dominios
Datos	Migración completa incluyendo IDENTITY values
Binarios	Columnas `VARBINARY(MAX)` (imágenes de variedades y portainjertos)
Usuarios	Tabla `usuarios` con hashes bcrypt — se migran tal cual
Audit log	Se migra histórico completo de `audit_log`

1.3 Audiencia

- **DBA / Administrador de BD**: Ejecutar scripts DDL y validar integridad
- **DevOps**: Configurar conexiones, ejecutar pipeline de migración
- **Desarrollador Backend**: Verificar compatibilidad de modelos SQLModel
- **Líder Técnico**: Aprobar ventana de migración y plan de rollback

1.4 Prerrequisitos

1. Python 3.11+ instalado con acceso a `pip`
2. `pyodbc` instalado (`pip install pyodbc`)
3. ODBC Driver 18 for SQL Server instalado en la máquina que ejecuta la migración
4. Credenciales de lectura en la BD origen (SQL Server Azure)
5. Credenciales de escritura (con permisos `db_owner`) en la BD destino
6. Conectividad de red entre la máquina de migración y ambos servidores
7. Ventana de mantenimiento acordada (sistema en modo solo lectura o detenido)

Advertencia

La migración debe ejecutarse con el sistema **detenido o en modo solo lectura** para garantizar consistencia. Cualquier escritura durante la migración puede causar pérdida de datos o inconsistencias en foreign keys.

Capítulo 2

Inventario de Tablas

El sistema contiene **52 tablas** organizadas en 6 dominios funcionales. Cada tabla usa `INT IDENTITY` como primary key (no UUID).

2.1 Resumen por dominio

Cuadro 2.1: Distribución de tablas por dominio

Dominio	Descripción	Tablas
Maestras / Lookup	Tablas de referencia y configuración	24
TestBlock	Infraestructura experimental	5
Inventario	Stock de vivero y plantas	7
Laboratorio / Calidad	Mediciones y clasificación	7
Análisis / Alertas	Paquetes tecnológicos y alertas	3
Sistema	Usuarios, roles, auditoría	6
Total		**52**

2.2 Catálogo completo de tablas

Cuadro 2.2: Catálogo completo de tablas del sistema

#	Tabla	PK	Dominio	FKs
1	paises	id_pais	Maestras	0
2	regiones	id_region	Maestras	0
3	comunas	id_comuna	Maestras	1

Continúa en la siguiente página

#	Tabla	PK	Dominio	FKs
4	campos	id_campo	Maestras	0
5	cuarteles	id_cuartel	Maestras	1
6	especies	id_especie	Maestras	0
7	portainjertos	id_portainjerto	Maestras	0
8	pmg	id_pmg	Maestras	0
9	pmg_especies	id_pmg_especie	Maestras	2
10	portainjerto_especies	id_pe	Maestras	2
11	origenes	id_origen	Maestras	0
12	viveros	id_vivero	Maestras	1
13	vivero_pmg	id_vp	Maestras	2
14	colores	id_color	Maestras	0
15	susceptibilidades	id_suscept	Maestras	0
16	variedades	id_variedad	Maestras	6
17	variedad_susceptibilidades	id_vs	Maestras	2
18	tipos_labor	id_labor	Maestras	0
19	estados_fenologicos	id_estado	Maestras	1
20	estados_planta	id_estado	Maestras	0
21	temporadas	id_temporada	Maestras	0
22	bodegas	id_bodega	Maestras	0
23	catalogos	id	Maestras	0
24	correlativos	id	Maestras	0
25	centros_costo	id	TestBlock	1
26	marcos_plantacion	id	TestBlock	0
27	testblocks	id_testblock	TestBlock	4
28	testblock_hileras	id_hilera	TestBlock	2
29	posiciones_testblock	id_posicion	TestBlock	6
30	inventario_vivero	id_inventario	Inventario	5
31	movimientos_inventario	id_movimiento	Inventario	2
32	plantas	id_planta	Inventario	6
33	inventario_testblock	id_inventario_tb	Inventario	2
34	guias_despacho	id_guia	Inventario	2
35	historial_posicion	id_historial	Inventario	2
36	asignaciones_testblock	id_asignacion	Inventario	2
37	mediciones_laboratorio	id_medicion	Laboratorio	6
38	clasificacion_cluster	id_clasificacion	Laboratorio	1
39	umbrales_calidad	id_umbral	Laboratorio	1
40	registros_fenologicos	id_registro	Laboratorio	3
41	ejecucion_labores	id_ejecucion	Laboratorio	3
42	detalles_labor	id_detalle	Laboratorio	1
43	evidencia_labores	id_evidencia	Laboratorio	1
44	paquete_tecnologico	id_paquete	Análisis	1
45	alertas	id_alerta	Análisis	1
46	reglas_alerta	id_regla	Análisis	0

Continúa en la siguiente página

#	**Tabla**	**PK**	**Dominio**	**FKs**
47	usuarios	id_usuario	Sistema	0
48	roles	id_rol	Sistema	0
49	audit_log	id_log	Sistema	0
50	variedades_log	id_log	Sistema	1
51	bitacora_variedades	id_entrada	Sistema	2
52	defectos	id	Sistema	0
	defectos_variedades	id	Sistema	2

Capítulo 3

Diagrama de Dependencias

3.1 Grafo de foreign keys

Las tablas deben crearse y cargarse en orden topológico para respetar las restricciones de foreign key. El grafo de dependencias define 5 niveles:

Cuadro 3.1: Orden topológico de creación y carga

Nivel	Tablas (sin dependencias pendientes)
0	paises, regiones, campos, especies, portainjertos, pmg, origenes, colores, susceptibilidades, tipos_labor, estados_planta, temporadas, bodegas, catalogos, correlativos, marcos_plantacion, reglas_alerta, roles, defectos
1	comunas, cuarteles, pmg_especies, portainjerto_especies, viveros, estados_fenologicos, centros_costo, usuarios, variedades
2	vivero_pmg, variedad_susceptibilidades, testblocks, inventario_vivero, detalles_labor, variedades_log, defectos_variedades
3	testblock_hileras, posiciones_testblock, plantas, inventario_testblock, guias_despacho, paquete_tecnologico, bitacora_variedades, asignaciones_testblock
4	movimientos_inventario, historial_posicion, mediciones_laboratorio, ejecucion_labores, registros_fenologicos, alertas
5	clasificacion_cluster, evidencia_labores, audit_log

Informacion

El nivel 0 contiene tablas sin foreign keys — pueden crearse y cargarse en cualquier orden entre ellas. Cada nivel posterior depende exclusivamente de los niveles anteriores.

3.2 Diagrama ER simplificado

```
campos  1:N  cuarteles

   1:N  testblocks  1:1

                 1:N  posiciones_testblock

                              1:1  plantas
                                          1:N  mediciones_laboratorio
                                          1:N  registros_fenologicos
                                          1:N  ejecucion_labores
                                          1:N  historial_posicion

                              N:1  variedades  N:1  especies

                                            N:1  pmg
                                            N:1  origenes

                 N:1  marcos_plantacion

inventario_vivero  1:N  movimientos_inventario
                   1:N  inventario_testblock

mediciones_laboratorio  1:1  clasificacion_cluster

ejecucion_labores  1:N  evidencia_labores
```

Capítulo 4

Arquitectura de la Migración

4.1 Estrategia general

Se utiliza **Python + pyodbc** como motor de migración, con scripts T-SQL versionados para la creación de estructura. La razón de esta elección sobre Flyway:

Cuadro 4.1: Comparación Flyway vs Python

Criterio	**Flyway**	**Python + pyodbc**
Dependencia extra	Java Runtime	Ya en el stack (FastAPI)
Transformación de datos	Limitada (solo SQL)	Total (pandas, validaciones)
Manejo de IDENTITY	Manual	Automatizado
Manejo de VARBINARY	Complejo	Nativo (bytes)
Integración CI/CD	Plugin adicional	Script en GitHub Actions
Scripts existentes	No	Sí (`load_data.py`, etc.)

4.2 Componentes del sistema de migración

```
backend/scripts/migration/
  config.py               # Conexiones origen/destino
  migrate.py              # Orquestador principal
  001_create_tables.sql   # DDL completo (52 tablas)
  002_create_indexes.sql  # Indices y constraints
  003_seed_data.sql       # Datos maestros (opcional)
  export_data.py          # Extrae datos del origen
  import_data.py          # Carga datos al destino
  validate.py             # Validacion post-migracion
```

Listing 4.1: Estructura de archivos de migración

4.3 Flujo de ejecución

1. **Preparación**: Configurar `config.py` con connection strings
2. **Crear estructura**: Ejecutar `001_create_tables.sql` en destino
3. **Crear índices**: Ejecutar `002_create_indexes.sql` en destino
4. **Exportar datos**: Ejecutar `export_data.py` (genera archivos intermedios o migra directo)
5. **Importar datos**: Ejecutar `import_data.py` (respetando orden topológico)
6. **Validar**: Ejecutar `validate.py` (conteos, checksums, integridad FK)
7. **Aceptar o Rollback**: Según resultados de validación

Capítulo 5

Script DDL — Creación de Tablas

A continuación se presenta el DDL completo para crear las 52 tablas en la base de datos destino. Las tablas están ordenadas por nivel de dependencia (nivel 0 primero).

Peligro

Ejecutar este script en una base de datos **vacía**. Si la BD destino ya tiene tablas con estos nombres, el script fallará. Use `DROP TABLE IF EXISTS` solo si está seguro de que no hay datos valiosos.

5.1 Nivel 0 — Tablas sin dependencias

```sql
CREATE TABLE paises (
    id_pais           INT IDENTITY(1,1) PRIMARY KEY,
    codigo            VARCHAR(10) NOT NULL,
    nombre            NVARCHAR(100) NOT NULL,
    nombre_en         NVARCHAR(100) NULL,
    orden             INT DEFAULT 0,
    activo            BIT DEFAULT 1,
    fecha_creacion    DATETIME DEFAULT GETDATE(),
    CONSTRAINT UQ_paises_codigo UNIQUE (codigo)
);
```

Listing 5.1: paises

```sql
CREATE TABLE regiones (
    id_region         INT IDENTITY(1,1) PRIMARY KEY,
    codigo            VARCHAR(5) NOT NULL,
    nombre            NVARCHAR(100) NOT NULL,
    numero            INT NULL,
    orden             INT NULL,
    activo            BIT DEFAULT 1,
    fecha_creacion    DATETIME DEFAULT GETDATE(),
```

```
    CONSTRAINT UQ_regiones_codigo UNIQUE (codigo)
);
```

Listing 5.2: regiones

```
CREATE TABLE campos (
    id_campo                INT IDENTITY(1,1) PRIMARY KEY,
    codigo                  VARCHAR(20) NOT NULL,
    nombre                  NVARCHAR(100) NOT NULL,
    ubicacion               NVARCHAR(200) NULL,
    comuna                  NVARCHAR(100) NULL,
    region                  NVARCHAR(100) NULL,
    direccion               NVARCHAR(200) NULL,
    responsable             NVARCHAR(100) NULL,
    hectareas               DECIMAL(10,2) NULL,
    latitud                 DECIMAL(10,7) NULL,
    longitud                DECIMAL(10,7) NULL,
    activo                  BIT DEFAULT 1,
    fecha_creacion          DATETIME DEFAULT GETDATE(),
    fecha_modificacion      DATETIME NULL,
    usuario_creacion        NVARCHAR(100) NULL,
    usuario_modificacion    NVARCHAR(100) NULL,
    CONSTRAINT UQ_campos_codigo UNIQUE (codigo)
);
```

Listing 5.3: campos

```
CREATE TABLE especies (
    id_especie              INT IDENTITY(1,1) PRIMARY KEY,
    codigo                  VARCHAR(10) NOT NULL,
    nombre                  NVARCHAR(50) NOT NULL,
    nombre_cientifico       NVARCHAR(100) NULL,
    emoji                   NVARCHAR(10) NULL,
    color_hex               VARCHAR(7) NULL,
    activo                  BIT DEFAULT 1,
    fecha_creacion          DATETIME DEFAULT GETDATE(),
    usuario_creacion        NVARCHAR(100) NULL,
    fecha_modificacion      DATETIME NULL,
    usuario_modificacion    NVARCHAR(100) NULL,
    CONSTRAINT UQ_especies_codigo UNIQUE (codigo)
);
```

Listing 5.4: especies

```
CREATE TABLE portainjertos (
    id_portainjerto         INT IDENTITY(1,1) PRIMARY KEY,
    codigo                  VARCHAR(20) NOT NULL,
    nombre                  NVARCHAR(50) NOT NULL,
    vigor                   VARCHAR(20) NULL,
    compatibilidad          NVARCHAR(MAX) NULL,
    origen                  NVARCHAR(100) NULL,
```

```
    caracteristicas        NVARCHAR(MAX) NULL,
    cruce                  NVARCHAR(200) NULL,
    especie                NVARCHAR(50) NULL,
    tipo                   NVARCHAR(50) NULL,
    patron                 NVARCHAR(100) NULL,
    propagacion            NVARCHAR(100) NULL,
    obtentor               NVARCHAR(100) NULL,
    sensibilidad           NVARCHAR(200) NULL,
    susceptibilidades      NVARCHAR(500) NULL,
    ventajas               NVARCHAR(500) NULL,
    notas                  NVARCHAR(MAX) NULL,
    imagen                 VARBINARY(MAX) NULL,
    activo                 BIT DEFAULT 1,
    fecha_creacion         DATETIME DEFAULT GETDATE(),
    usuario_creacion       NVARCHAR(100) NULL,
    fecha_modificacion     DATETIME NULL,
    usuario_modificacion   NVARCHAR(100) NULL,
    CONSTRAINT UQ_portainjertos_codigo UNIQUE (codigo)
);
```

Listing 5.5: portainjertos

```
CREATE TABLE pmg (
    id_pmg                 INT IDENTITY(1,1) PRIMARY KEY,
    codigo                 VARCHAR(20) NOT NULL,
    nombre                 NVARCHAR(100) NOT NULL,
    licenciante            NVARCHAR(100) NULL,
    pais_origen            VARCHAR(50) NULL,
    pais                   VARCHAR(50) NULL,
    ciudad                 NVARCHAR(100) NULL,
    email                  NVARCHAR(100) NULL,
    telefono               VARCHAR(30) NULL,
    direccion              NVARCHAR(200) NULL,
    contacto               NVARCHAR(200) NULL,
    notas                  NVARCHAR(MAX) NULL,
    contacto_nombre        NVARCHAR(200) NULL,
    contacto_email         NVARCHAR(100) NULL,
    contacto_telefono      NVARCHAR(50) NULL,
    viveros_chile          NVARCHAR(500) NULL,
    activo                 BIT DEFAULT 1,
    fecha_creacion         DATETIME DEFAULT GETDATE(),
    usuario_creacion       NVARCHAR(100) NULL,
    usuario_modificacion   NVARCHAR(100) NULL,
    fecha_modificacion     DATETIME NULL,
    CONSTRAINT UQ_pmg_codigo UNIQUE (codigo)
);
```

Listing 5.6: pmg

```
CREATE TABLE origenes (
```

```sql
    id_origen               INT IDENTITY(1,1) PRIMARY KEY,
    codigo                  VARCHAR(20) NOT NULL,
    nombre                  NVARCHAR(100) NOT NULL,
    pais                    VARCHAR(50) NULL,
    tipo                    VARCHAR(30) NULL,
    contacto                NVARCHAR(200) NULL,
    notas                   NVARCHAR(MAX) NULL,
    activo                  BIT DEFAULT 1,
    fecha_creacion          DATETIME DEFAULT GETDATE(),
    usuario_creacion        NVARCHAR(100) NULL,
    fecha_modificacion      DATETIME NULL,
    usuario_modificacion    NVARCHAR(100) NULL,
    CONSTRAINT UQ_origenes_codigo UNIQUE (codigo)
);
```

Listing 5.7: origenes

```sql
CREATE TABLE colores (
    id_color                INT IDENTITY(1,1) PRIMARY KEY,
    codigo                  VARCHAR(20) NOT NULL,
    nombre                  NVARCHAR(50) NOT NULL,
    tipo                    VARCHAR(20) NOT NULL,
    aplica_especie          NVARCHAR(100) NULL,
    color_hex               VARCHAR(7) NULL,
    activo                  BIT DEFAULT 1,
    fecha_creacion          DATETIME DEFAULT GETDATE(),
    usuario_creacion        NVARCHAR(100) NULL,
    fecha_modificacion      DATETIME NULL,
    usuario_modificacion    NVARCHAR(100) NULL
);
```

Listing 5.8: colores

```sql
CREATE TABLE susceptibilidades (
    id_suscept              INT IDENTITY(1,1) PRIMARY KEY,
    codigo                  VARCHAR(20) NOT NULL,
    nombre                  NVARCHAR(100) NOT NULL,
    nombre_en               NVARCHAR(100) NULL,
    descripcion             NVARCHAR(200) NULL,
    categoria               NVARCHAR(50) NULL,
    severidad               VARCHAR(20) DEFAULT 'media',
    orden                   INT DEFAULT 0,
    activo                  BIT DEFAULT 1,
    fecha_creacion          DATETIME DEFAULT GETDATE(),
    usuario_creacion        NVARCHAR(100) NULL,
    fecha_modificacion      DATETIME NULL,
    usuario_modificacion    NVARCHAR(100) NULL,
    CONSTRAINT UQ_suscept_codigo UNIQUE (codigo)
);
```

Listing 5.9: susceptibilidades

```sql
CREATE TABLE tipos_labor (
    id_labor                INT IDENTITY(1,1) PRIMARY KEY,
    codigo                  VARCHAR(20) NOT NULL,
    categoria               VARCHAR(50) NOT NULL,
    nombre                  NVARCHAR(100) NOT NULL,
    descripcion             NVARCHAR(MAX) NULL,
    aplica_especies         NVARCHAR(200) NULL,
    aplica_a                NVARCHAR(100) NULL,
    frecuencia              NVARCHAR(50) NULL,
    activo                  BIT DEFAULT 1,
    fecha_creacion          DATETIME DEFAULT GETDATE(),
    usuario_creacion        NVARCHAR(100) NULL,
    fecha_modificacion      DATETIME NULL,
    usuario_modificacion    NVARCHAR(100) NULL,
    CONSTRAINT UQ_tipos_labor_codigo UNIQUE (codigo)
);
```

Listing 5.10: tipos_labor

```sql
CREATE TABLE estados_planta (
    id_estado               INT IDENTITY(1,1) PRIMARY KEY,
    codigo                  VARCHAR(20) NOT NULL,
    nombre                  NVARCHAR(50) NOT NULL,
    descripcion             NVARCHAR(200) NULL,
    color_hex               VARCHAR(7) NOT NULL,
    icono                   VARCHAR(50) NULL,
    requiere_foto           BIT DEFAULT 0,
    es_final                BIT DEFAULT 0,
    orden                   INT DEFAULT 0,
    activo                  BIT DEFAULT 1,
    fecha_creacion          DATETIME DEFAULT GETDATE(),
    usuario_creacion        NVARCHAR(100) NULL,
    fecha_modificacion      DATETIME NULL,
    usuario_modificacion    NVARCHAR(100) NULL,
    CONSTRAINT UQ_estados_planta_codigo UNIQUE (codigo)
);
```

Listing 5.11: estados_planta

```sql
CREATE TABLE temporadas (
    id_temporada            INT IDENTITY(1,1) PRIMARY KEY,
    codigo                  NVARCHAR(20) NOT NULL,
    nombre                  NVARCHAR(50) NOT NULL,
    fecha_inicio            DATE NULL,
    fecha_fin               DATE NULL,
    estado                  NVARCHAR(20) DEFAULT 'activa',
    notas                   NVARCHAR(MAX) NULL,
    activo                  BIT DEFAULT 1,
    fecha_creacion          DATETIME DEFAULT GETDATE(),
    CONSTRAINT UQ_temporadas_codigo UNIQUE (codigo)
```

```
);
```

Listing 5.12: temporadas

```sql
CREATE TABLE bodegas (
    id_bodega               INT IDENTITY(1,1) PRIMARY KEY,
    codigo                  NVARCHAR(20) NOT NULL,
    nombre                  NVARCHAR(100) NOT NULL,
    ubicacion               NVARCHAR(200) NULL,
    responsable             NVARCHAR(100) NULL,
    activo                  BIT DEFAULT 1,
    fecha_creacion          DATETIME DEFAULT GETDATE()
);
```

Listing 5.13: bodegas

```sql
CREATE TABLE catalogos (
    id                      INT IDENTITY(1,1) PRIMARY KEY,
    tipo                    NVARCHAR(100) NOT NULL,
    valor                   NVARCHAR(200) NOT NULL,
    descripcion             NVARCHAR(500) NULL,
    orden                   INT DEFAULT 0,
    activo                  BIT DEFAULT 1,
    fecha_creacion          DATETIME DEFAULT GETDATE()
);
```

Listing 5.14: catalogos

```sql
CREATE TABLE correlativos (
    id                      INT IDENTITY(1,1) PRIMARY KEY,
    tipo                    VARCHAR(30) NOT NULL,
    prefijo                 VARCHAR(10) NOT NULL,
    ultimo_numero           INT DEFAULT 0,
    formato                 VARCHAR(50) NULL,
    fecha_modificacion      DATETIME NULL,
    CONSTRAINT UQ_correlativos_tipo UNIQUE (tipo)
);
```

Listing 5.15: correlativos

```sql
CREATE TABLE marcos_plantacion (
    id                      INT IDENTITY(1,1) PRIMARY KEY,
    codigo                  NVARCHAR(50) NOT NULL,
    nombre                  NVARCHAR(200) NOT NULL,
    distancia_hilera        DECIMAL(5,2) NULL,
    distancia_planta        DECIMAL(5,2) NULL,
    sistema_conduccion      NVARCHAR(100) NULL,
    descripcion             NVARCHAR(500) NULL,
    dist_entre_hileras      DECIMAL(10,2) NULL,
    dist_entre_plantas      DECIMAL(10,2) NULL,
    plantas_hectarea        INT NULL,
```

```
    conduccion              NVARCHAR(100) NULL,
    especie_recomendada     NVARCHAR(100) NULL,
    activo                  BIT DEFAULT 1,
    fecha_creacion          DATETIME DEFAULT GETDATE(),
    usuario_creacion        NVARCHAR(100) NULL,
    fecha_modificacion      DATETIME NULL,
    usuario_modificacion    NVARCHAR(100) NULL,
    id_marco                INT NULL,
    predeterminado          BIT NULL,
    notas                   NVARCHAR(MAX) NULL,
    CONSTRAINT UQ_marcos_codigo UNIQUE (codigo)
);
```

Listing 5.16: marcos_plantacion

```
CREATE TABLE reglas_alerta (
    id_regla                INT IDENTITY(1,1) PRIMARY KEY,
    codigo                  VARCHAR(20) NOT NULL,
    nombre                  NVARCHAR(100) NULL,
    descripcion             NVARCHAR(MAX) NULL,
    tipo                    VARCHAR(50) NULL,
    condicion               NVARCHAR(MAX) NULL,
    prioridad_resultado     VARCHAR(20) NULL,
    activo                  BIT DEFAULT 1,
    fecha_creacion          DATETIME DEFAULT GETDATE(),
    CONSTRAINT UQ_reglas_alerta_codigo UNIQUE (codigo)
);
```

Listing 5.17: reglas_alerta

```
CREATE TABLE roles (
    id_rol                  INT IDENTITY(1,1) PRIMARY KEY,
    nombre                  VARCHAR(50) NOT NULL,
    descripcion             NVARCHAR(200) NULL,
    permisos                NVARCHAR(MAX) NULL,
    activo                  BIT DEFAULT 1,
    CONSTRAINT UQ_roles_nombre UNIQUE (nombre)
);
```

Listing 5.18: roles

```
CREATE TABLE defectos (
    id                      INT IDENTITY(1,1) PRIMARY KEY,
    codigo                  VARCHAR(20) NULL,
    nombre                  VARCHAR(100) NOT NULL,
    nombre_en               VARCHAR(100) NULL,
    categoria               VARCHAR(50) NULL,
    activo                  BIT DEFAULT 1,
    orden                   INT NULL,
    created_at              DATETIME NULL,
    updated_at              DATETIME NULL,
```

```
    imagen_url              NVARCHAR(500) NULL,
    imagen                  VARBINARY(MAX) NULL
);
```

Listing 5.19: defectos

5.2 Nivel 1 — Tablas con dependencias de nivel 0

```
CREATE TABLE comunas (
    id_comuna               INT IDENTITY(1,1) PRIMARY KEY,
    nombre                  NVARCHAR(100) NOT NULL,
    id_region               INT NOT NULL
        CONSTRAINT FK_comunas_region FOREIGN KEY
        REFERENCES regiones(id_region),
    codigo_postal           VARCHAR(10) NULL,
    activo                  BIT DEFAULT 1,
    fecha_creacion          DATETIME DEFAULT GETDATE()
);
```

Listing 5.20: comunas

```
CREATE TABLE cuarteles (
    id_cuartel              INT IDENTITY(1,1) PRIMARY KEY,
    id_campo                INT NOT NULL
        CONSTRAINT FK_cuarteles_campo FOREIGN KEY
        REFERENCES campos(id_campo),
    codigo                  VARCHAR(20) NOT NULL,
    nombre                  NVARCHAR(100) NOT NULL,
    num_hileras             INT NULL,
    pos_por_hilera          INT NULL,
    es_testblock            BIT NULL,
    centro_costo            VARCHAR(50) NULL,
    activo                  BIT DEFAULT 1,
    fecha_creacion          DATETIME DEFAULT GETDATE(),
    hileras                 INT NULL,
    posiciones_por_hilera   INT NULL,
    sistema                 NVARCHAR(100) NULL,
    notas                   NVARCHAR(MAX) NULL,
    id_centro_costo         INT NULL,
    id_marco                INT NULL,
    latitud                 DECIMAL(10,6) NULL,
    longitud                DECIMAL(10,6) NULL,
    usuario_creacion        NVARCHAR(100) NULL,
    fecha_modificacion      DATETIME NULL,
    usuario_modificacion    NVARCHAR(100) NULL,
    superficie              DECIMAL(10,2) NULL,
    id_especie              INT NULL,
    especies_ids            NVARCHAR(MAX) NULL,
```

```
    layout_configurado     BIT NULL
);
```

Listing 5.21: cuarteles

```
CREATE TABLE usuarios (
    id_usuario             INT IDENTITY(1,1) PRIMARY KEY,
    username               VARCHAR(50) NOT NULL,
    nombre_completo        NVARCHAR(100) NOT NULL,
    email                  NVARCHAR(100) NULL,
    password_hash          VARCHAR(64) NULL,
    rol                    VARCHAR(30) NOT NULL,
    campos_asignados       NVARCHAR(MAX) NULL,
    activo                 BIT DEFAULT 1,
    ultimo_acceso          DATETIME NULL,
    fecha_creacion         DATETIME DEFAULT GETDATE(),
    CONSTRAINT UQ_usuarios_username UNIQUE (username)
);
```

Listing 5.22: usuarios

```
CREATE TABLE estados_fenologicos (
    id_estado              INT IDENTITY(1,1) PRIMARY KEY,
    id_especie             INT NOT NULL
        CONSTRAINT FK_estadosfenol_especie FOREIGN KEY
        REFERENCES especies(id_especie),
    codigo                 VARCHAR(20) NOT NULL,
    nombre                 NVARCHAR(50) NOT NULL,
    orden                  INT DEFAULT 0,
    descripcion            NVARCHAR(200) NULL,
    color_hex              VARCHAR(7) NULL,
    mes_orientativo        NVARCHAR(20) NULL,
    activo                 BIT DEFAULT 1,
    fecha_creacion         DATETIME DEFAULT GETDATE(),
    usuario_creacion       NVARCHAR(100) NULL,
    fecha_modificacion     DATETIME NULL,
    usuario_modificacion   NVARCHAR(100) NULL
);
```

Listing 5.23: estados_fenologicos

```
CREATE TABLE centros_costo (
    id                     INT IDENTITY(1,1) PRIMARY KEY,
    codigo                 NVARCHAR(50) NOT NULL,
    nombre                 NVARCHAR(200) NOT NULL,
    descripcion            NVARCHAR(500) NULL,
    id_campo               INT NULL
        CONSTRAINT FK_centroscosto_campo FOREIGN KEY
        REFERENCES campos(id_campo),
    responsable            NVARCHAR(200) NULL,
    presupuesto            DECIMAL(18,2) NULL,
```

```
    activo                  BIT DEFAULT 1,
    fecha_creacion          DATETIME DEFAULT GETDATE(),
    usuario_creacion        NVARCHAR(100) NULL,
    fecha_modificacion      DATETIME NULL,
    usuario_modificacion    NVARCHAR(100) NULL,
    CONSTRAINT UQ_centros_costo_codigo UNIQUE (codigo)
);
```

Listing 5.24: centros_costo

```
CREATE TABLE variedades (
    id_variedad             INT IDENTITY(1,1) PRIMARY KEY,
    id_especie              INT NOT NULL
        CONSTRAINT FK_variedades_especie FOREIGN KEY
        REFERENCES especies(id_especie),
    id_pmg                  INT NULL
        CONSTRAINT FK_variedades_pmg FOREIGN KEY
        REFERENCES pmg(id_pmg),
    id_origen               INT NULL
        CONSTRAINT FK_variedades_origen FOREIGN KEY
        REFERENCES origenes(id_origen),
    codigo                  VARCHAR(30) NOT NULL,
    nombre                  NVARCHAR(100) NOT NULL,
    nombre_corto            VARCHAR(10) NULL,
    nombre_comercial        NVARCHAR(100) NULL,
    tipo                    VARCHAR(20) NULL,
    origen                  NVARCHAR(100) NULL,
    anio_introduccion       INT NULL,
    epoca_cosecha           VARCHAR(20) NULL,
    epoca                   VARCHAR(20) NULL,
    vigor                   VARCHAR(20) NULL,
    req_frio_horas          INT NULL,
    req_frio                NVARCHAR(50) NULL,
    color_fruto             NVARCHAR(50) NULL,
    color_pulpa             NVARCHAR(50) NULL,
    id_color_fruto          INT NULL
        CONSTRAINT FK_variedades_colorfruto FOREIGN KEY
        REFERENCES colores(id_color),
    id_color_pulpa          INT NULL
        CONSTRAINT FK_variedades_colorpulpa FOREIGN KEY
        REFERENCES colores(id_color),
    id_color_cubrimiento    INT NULL
        CONSTRAINT FK_variedades_colorcubr FOREIGN KEY
        REFERENCES colores(id_color),
    calibre_esperado        VARCHAR(20) NULL,
    firmeza_esperada        VARCHAR(20) NULL,
    susceptibilidad         NVARCHAR(200) NULL,
    estado                  VARCHAR(20) NULL,
    fecha_ultima_visita     DATE NULL,
    proxima_accion          NVARCHAR(200) NULL,
```

```
    observaciones          NVARCHAR(MAX) NULL,
    imagen                 VARBINARY(MAX) NULL,
    color                  NVARCHAR(50) NULL,
    fertilidad             NVARCHAR(50) NULL,
    alelos                 NVARCHAR(200) NULL,
    polinizantes           NVARCHAR(MAX) NULL,
    portainjertos_recomendados NVARCHAR(MAX) NULL,
    familia_genetica       NVARCHAR(100) NULL,
    recomendaciones        NVARCHAR(MAX) NULL,
    fecha_cosecha_ref      NVARCHAR(50) NULL,
    auto_fertil            BIT NULL,
    activo                 BIT DEFAULT 1,
    fecha_creacion         DATETIME DEFAULT GETDATE(),
    fecha_modificacion     DATETIME NULL,
    usuario_creacion       NVARCHAR(100) NULL,
    usuario_modificacion   NVARCHAR(100) NULL,
    CONSTRAINT UQ_variedades_codigo UNIQUE (codigo)
);
```

Listing 5.25: variedades

```
CREATE TABLE pmg_especies (
    id_pmg_especie         INT IDENTITY(1,1) PRIMARY KEY,
    id_pmg                 INT NULL
        CONSTRAINT FK_pmgespecies_pmg FOREIGN KEY
        REFERENCES pmg(id_pmg),
    id_especie             INT NULL
        CONSTRAINT FK_pmgespecies_especie FOREIGN KEY
        REFERENCES especies(id_especie),
    activo                 BIT DEFAULT 1,
    fecha_creacion         DATETIME DEFAULT GETDATE(),
    CONSTRAINT UQ_pmg_especies UNIQUE (id_pmg, id_especie)
);
```

Listing 5.26: pmg_especies

```
CREATE TABLE portainjerto_especies (
    id_pe                  INT IDENTITY(1,1) PRIMARY KEY,
    id_portainjerto        INT NOT NULL
        CONSTRAINT FK_portesp_portainjerto FOREIGN KEY
        REFERENCES portainjertos(id_portainjerto),
    id_especie             INT NOT NULL
        CONSTRAINT FK_portesp_especie FOREIGN KEY
        REFERENCES especies(id_especie),
    activo                 BIT DEFAULT 1,
    fecha_creacion         DATETIME DEFAULT GETDATE()
);
```

Listing 5.27: portainjerto_especies

```sql
CREATE TABLE viveros (
    id_vivero               INT IDENTITY(1,1) PRIMARY KEY,
    codigo                  VARCHAR(20) NOT NULL,
    nombre                  NVARCHAR(100) NOT NULL,
    id_pmg                  INT NULL
        CONSTRAINT FK_viveros_pmg FOREIGN KEY
        REFERENCES pmg(id_pmg),
    representante           NVARCHAR(100) NULL,
    telefono                VARCHAR(30) NULL,
    email                   NVARCHAR(100) NULL,
    direccion               NVARCHAR(200) NULL,
    comuna                  NVARCHAR(100) NULL,
    region                  NVARCHAR(100) NULL,
    activo                  BIT DEFAULT 1,
    fecha_creacion          DATETIME DEFAULT GETDATE(),
    usuario_creacion        NVARCHAR(100) NULL,
    fecha_modificacion      DATETIME NULL,
    usuario_modificacion    NVARCHAR(100) NULL,
    CONSTRAINT UQ_viveros_codigo UNIQUE (codigo)
);
```

Listing 5.28: viveros

5.3 Nivel 2 — Tablas con dependencias de nivel 0–1

```sql
CREATE TABLE vivero_pmg (
    id_vp                   INT IDENTITY(1,1) PRIMARY KEY,
    id_vivero               INT NOT NULL
        CONSTRAINT FK_viveropmg_vivero FOREIGN KEY
        REFERENCES viveros(id_vivero),
    id_pmg                  INT NOT NULL
        CONSTRAINT FK_viveropmg_pmg FOREIGN KEY
        REFERENCES pmg(id_pmg),
    activo                  BIT DEFAULT 1,
    fecha_creacion          DATETIME DEFAULT GETDATE()
);
```

Listing 5.29: vivero_pmg

```sql
CREATE TABLE variedad_susceptibilidades (
    id_vs                   INT IDENTITY(1,1) PRIMARY KEY,
    id_variedad             INT NOT NULL
        CONSTRAINT FK_varsuscept_variedad FOREIGN KEY
        REFERENCES variedades(id_variedad),
    id_suscept              INT NOT NULL
        CONSTRAINT FK_varsuscept_suscept FOREIGN KEY
        REFERENCES susceptibilidades(id_suscept),
```

```
    nivel                   VARCHAR(20) NULL,
    notas                   NVARCHAR(200) NULL,
    CONSTRAINT UQ_variedad_suscept UNIQUE (id_variedad, id_suscept)
);
```

Listing 5.30: variedad_susceptibilidades

```
CREATE TABLE testblocks (
    id_testblock            INT IDENTITY(1,1) PRIMARY KEY,
    codigo                  VARCHAR(20) NOT NULL,
    nombre                  NVARCHAR(100) NOT NULL,
    id_campo                INT NOT NULL
        CONSTRAINT FK_testblocks_campo FOREIGN KEY
        REFERENCES campos(id_campo),
    id_centro_costo         INT NULL
        CONSTRAINT FK_testblocks_centrocosto FOREIGN KEY
        REFERENCES centros_costo(id),
    id_cuartel              INT NULL
        CONSTRAINT FK_testblocks_cuartel FOREIGN KEY
        REFERENCES cuarteles(id_cuartel),
    id_marco                INT NULL
        CONSTRAINT FK_testblocks_marco FOREIGN KEY
        REFERENCES marcos_plantacion(id),
    num_hileras             INT NULL,
    posiciones_por_hilera INT NULL,
    total_posiciones        INT NULL,
    latitud                 DECIMAL(10,7) NULL,
    longitud                DECIMAL(10,7) NULL,
    estado                  VARCHAR(20) DEFAULT 'activo',
    fecha_creacion_tb       DATETIME NULL,
    temporada_inicio        VARCHAR(10) NULL,
    notas                   NVARCHAR(MAX) NULL,
    activo                  BIT DEFAULT 1,
    fecha_creacion          DATETIME DEFAULT GETDATE(),
    fecha_modificacion      DATETIME NULL,
    CONSTRAINT UQ_testblocks_codigo UNIQUE (codigo)
);
```

Listing 5.31: testblocks

```
CREATE TABLE inventario_vivero (
    id_inventario            INT IDENTITY(1,1) PRIMARY KEY,
    codigo_lote              VARCHAR(50) NOT NULL,
    id_variedad              INT NULL
        CONSTRAINT FK_invvivero_variedad FOREIGN KEY
        REFERENCES variedades(id_variedad),
    id_portainjerto          INT NULL
        CONSTRAINT FK_invvivero_portainjerto FOREIGN KEY
        REFERENCES portainjertos(id_portainjerto),
    id_vivero                INT NULL
```

```
        CONSTRAINT FK_invvivero_vivero FOREIGN KEY
        REFERENCES viveros(id_vivero),
    id_especie              INT NULL
        CONSTRAINT FK_invvivero_especie FOREIGN KEY
        REFERENCES especies(id_especie),
    id_pmg                  INT NULL
        CONSTRAINT FK_invvivero_pmg FOREIGN KEY
        REFERENCES pmg(id_pmg),
    id_bodega               INT NULL
        CONSTRAINT FK_invvivero_bodega FOREIGN KEY
        REFERENCES bodegas(id_bodega),
    tipo_planta             NVARCHAR(50) NULL,
    tipo_injertacion        NVARCHAR(50) NULL,
    tipo_patron             NVARCHAR(50) NULL,
    ubicacion               NVARCHAR(200) NULL,
    cantidad_inicial        INT NOT NULL,
    cantidad_actual         INT NOT NULL,
    cantidad_minima         INT DEFAULT 0,
    cantidad_comprometida   INT DEFAULT 0,
    fecha_ingreso           DATE NOT NULL,
    ano_plantacion          INT NULL,
    origen                  NVARCHAR(100) NULL,
    estado                  VARCHAR(20) DEFAULT 'disponible',
    observaciones           NVARCHAR(MAX) NULL,
    fecha_creacion          DATETIME DEFAULT GETDATE(),
    fecha_modificacion      DATETIME NULL,
    CONSTRAINT UQ_invvivero_lote UNIQUE (codigo_lote)
);
```

Listing 5.32: inventario_vivero

```
CREATE TABLE detalles_labor (
    id_detalle              INT IDENTITY(1,1) PRIMARY KEY,
    id_labor                INT NOT NULL
        CONSTRAINT FK_detalleslabor_labor FOREIGN KEY
        REFERENCES tipos_labor(id_labor),
    descripcion             NVARCHAR(500) NOT NULL,
    aplica_especie          NVARCHAR(100) NULL,
    orden                   INT DEFAULT 0,
    es_checklist            BIT DEFAULT 1,
    activo                  BIT DEFAULT 1,
    fecha_creacion          DATETIME DEFAULT GETDATE(),
    usuario_creacion        NVARCHAR(100) NULL,
    fecha_modificacion      DATETIME NULL,
    usuario_modificacion    NVARCHAR(100) NULL
);
```

Listing 5.33: detalles_labor

```
CREATE TABLE variedades_log (
```

```
    id_log                  INT IDENTITY(1,1) PRIMARY KEY,
    id_variedad             INT NOT NULL
        CONSTRAINT FK_varlog_variedad FOREIGN KEY
        REFERENCES variedades(id_variedad),
    accion                  VARCHAR(50) NOT NULL,
    campo_modificado        VARCHAR(100) NULL,
    valor_anterior          NVARCHAR(500) NULL,
    valor_nuevo             NVARCHAR(500) NULL,
    usuario                 VARCHAR(50) NULL,
    fecha                   DATETIME NULL,
    notas                   NVARCHAR(500) NULL
);
```

Listing 5.34: variedades_log

```
CREATE TABLE defectos_variedades (
    id                      INT IDENTITY(1,1) PRIMARY KEY,
    id_variedad             INT NOT NULL
        CONSTRAINT FK_defvar_variedad FOREIGN KEY
        REFERENCES variedades(id_variedad),
    id_defecto              INT NOT NULL
        CONSTRAINT FK_defvar_defecto FOREIGN KEY
        REFERENCES defectos(id),
    created_at              DATETIME NULL
);
```

Listing 5.35: defectos_variedades

5.4 Nivel 3 — Tablas con dependencias de nivel 0–2

```
CREATE TABLE testblock_hileras (
    id_hilera                   INT IDENTITY(1,1) PRIMARY KEY,
    id_cuartel                  INT NOT NULL
        CONSTRAINT FK_tbhileras_cuartel FOREIGN KEY
        REFERENCES cuarteles(id_cuartel),
    numero_hilera               INT NOT NULL,
    total_posiciones            INT NOT NULL,
    portainjerto_default_id     INT NULL
        CONSTRAINT FK_tbhileras_portainjerto FOREIGN KEY
        REFERENCES portainjertos(id_portainjerto),
    conduccion                  NVARCHAR(50) NULL,
    marco_plantacion            VARCHAR(20) NULL,
    activo                      BIT DEFAULT 1,
    fecha_creacion              DATETIME DEFAULT GETDATE(),
    usuario_creacion            NVARCHAR(50) NULL
);
```

Listing 5.36: testblock_hileras

```sql
CREATE TABLE posiciones_testblock (
    id_posicion            INT IDENTITY(1,1) PRIMARY KEY,
    codigo_unico           VARCHAR(30) NOT NULL,
    id_cuartel             INT NULL
        CONSTRAINT FK_postb_cuartel FOREIGN KEY
        REFERENCES cuarteles(id_cuartel),
    id_testblock           INT NULL
        CONSTRAINT FK_postb_testblock FOREIGN KEY
        REFERENCES testblocks(id_testblock),
    id_variedad            INT NULL
        CONSTRAINT FK_postb_variedad FOREIGN KEY
        REFERENCES variedades(id_variedad),
    id_portainjerto        INT NULL
        CONSTRAINT FK_postb_portainjerto FOREIGN KEY
        REFERENCES portainjertos(id_portainjerto),
    id_pmg                 INT NULL
        CONSTRAINT FK_postb_pmg FOREIGN KEY
        REFERENCES pmg(id_pmg),
    id_lote                INT NULL
        CONSTRAINT FK_postb_lote FOREIGN KEY
        REFERENCES inventario_vivero(id_inventario),
    hilera                 INT NOT NULL,
    posicion               INT NOT NULL,
    fecha_plantacion       DATE NULL,
    fecha_alta             DATE NULL,
    fecha_baja             DATE NULL,
    estado                 VARCHAR(20) DEFAULT 'vacia',
    cluster_actual         INT NULL,
    motivo_baja            NVARCHAR(200) NULL,
    observaciones          NVARCHAR(MAX) NULL,
    codigo_qr              NVARCHAR(MAX) NULL,
    usuario_alta           NVARCHAR(50) NULL,
    usuario_baja           NVARCHAR(50) NULL,
    protegida              BIT DEFAULT 0,
    conduccion             NVARCHAR(50) NULL,
    marco_plantacion       VARCHAR(20) NULL,
    fecha_creacion         DATETIME DEFAULT GETDATE(),
    fecha_modificacion     DATETIME NULL,
    CONSTRAINT UQ_postb_codigo UNIQUE (codigo_unico),
    CONSTRAINT UQ_postb_cuartel_hilera_pos
        UNIQUE (id_cuartel, hilera, posicion)
);
```

Listing 5.37: posiciones_testblock

```sql
CREATE TABLE plantas (
    id_planta              INT IDENTITY(1,1) PRIMARY KEY,
    codigo                 VARCHAR(50) NULL,
    id_posicion            INT NULL
```

```
        CONSTRAINT FK_plantas_posicion FOREIGN KEY
        REFERENCES posiciones_testblock(id_posicion),
    id_variedad            INT NULL
        CONSTRAINT FK_plantas_variedad FOREIGN KEY
        REFERENCES variedades(id_variedad),
    id_portainjerto        INT NULL
        CONSTRAINT FK_plantas_portainjerto FOREIGN KEY
        REFERENCES portainjertos(id_portainjerto),
    id_especie             INT NULL
        CONSTRAINT FK_plantas_especie FOREIGN KEY
        REFERENCES especies(id_especie),
    id_pmg                 INT NULL
        CONSTRAINT FK_plantas_pmg FOREIGN KEY
        REFERENCES pmg(id_pmg),
    id_lote_origen         INT NULL
        CONSTRAINT FK_plantas_lote FOREIGN KEY
        REFERENCES inventario_vivero(id_inventario),
    condicion              VARCHAR(30) DEFAULT 'EN_EVALUACION',
    activa                 BIT DEFAULT 1,
    ano_plantacion         INT NULL,
    ano_injertacion        INT NULL,
    metodo_injertacion     NVARCHAR(50) NULL,
    tipo_patron            NVARCHAR(50) NULL,
    conduccion             NVARCHAR(50) NULL,
    marco_plantacion       VARCHAR(20) NULL,
    color_cubrimiento      NVARCHAR(50) NULL,
    color_pulpa            NVARCHAR(50) NULL,
    fecha_alta             DATE NULL,
    fecha_baja             DATE NULL,
    motivo_baja            NVARCHAR(200) NULL,
    observaciones          NVARCHAR(MAX) NULL,
    fecha_creacion         DATETIME DEFAULT GETDATE(),
    fecha_modificacion     DATETIME NULL,
    usuario_creacion       NVARCHAR(50) NULL,
    usuario_modificacion   NVARCHAR(50) NULL,
    CONSTRAINT UQ_plantas_codigo UNIQUE (codigo)
);
```

Listing 5.38: plantas

```
CREATE TABLE inventario_testblock (
    id_inventario_tb       INT IDENTITY(1,1) PRIMARY KEY,
    id_inventario          INT NULL
        CONSTRAINT FK_invtb_inventario FOREIGN KEY
        REFERENCES inventario_vivero(id_inventario),
    id_cuartel             INT NULL
        CONSTRAINT FK_invtb_cuartel FOREIGN KEY
        REFERENCES cuarteles(id_cuartel),
    cantidad_asignada      INT NULL,
    cantidad_plantada      INT DEFAULT 0,
```

```
    estado                  VARCHAR(20) DEFAULT 'pendiente',
    fecha_despacho          DATE NULL,
    fecha_completado        DATE NULL,
    observaciones           NVARCHAR(MAX) NULL,
    usuario_creacion        NVARCHAR(50) NULL,
    fecha_creacion          DATETIME DEFAULT GETDATE()
);
```

Listing 5.39: inventario_testblock

```
CREATE TABLE guias_despacho (
    id_guia                 INT IDENTITY(1,1) PRIMARY KEY,
    numero_guia             VARCHAR(20) NOT NULL,
    id_bodega_origen        INT NOT NULL
        CONSTRAINT FK_guias_bodega FOREIGN KEY
        REFERENCES bodegas(id_bodega),
    id_testblock_destino    INT NOT NULL
        CONSTRAINT FK_guias_testblock FOREIGN KEY
        REFERENCES testblocks(id_testblock),
    estado                  VARCHAR(20) DEFAULT 'pendiente',
    total_plantas           INT NOT NULL,
    responsable             NVARCHAR(100) NULL,
    motivo                  NVARCHAR(500) NULL,
    usuario                 NVARCHAR(50) NULL,
    fecha_creacion          DATETIME DEFAULT GETDATE(),
    fecha_despacho          DATETIME NULL,
    fecha_recepcion         DATETIME NULL,
    notas                   NVARCHAR(MAX) NULL,
    activo                  BIT DEFAULT 1
);
```

Listing 5.40: guias_despacho

```
CREATE TABLE paquete_tecnologico (
    id_paquete              INT IDENTITY(1,1) PRIMARY KEY,
    id_variedad             INT NOT NULL
        CONSTRAINT FK_paqtec_variedad FOREIGN KEY
        REFERENCES variedades(id_variedad),
    temporada               VARCHAR(50) NOT NULL,
    total_posiciones        INT NULL,
    posiciones_evaluadas    INT NULL,
    cluster_predominante    INT NULL,
    brix_promedio           DECIMAL(5,2) NULL,
    brix_min                DECIMAL(5,2) NULL,
    brix_max                DECIMAL(5,2) NULL,
    firmeza_promedio        DECIMAL(5,2) NULL,
    acidez_promedio         DECIMAL(5,3) NULL,
    calibre_promedio        DECIMAL(5,2) NULL,
    score_promedio          DECIMAL(5,2) NULL,
    recomendacion           NVARCHAR(MAX) NULL,
```

```
    decision                VARCHAR(20) NULL,
    fecha_generacion        DATETIME NULL,
    CONSTRAINT UQ_paqtec_variedad_temp
        UNIQUE (id_variedad, temporada)
);
```

Listing 5.41: paquete_tecnologico

```
CREATE TABLE bitacora_variedades (
    id_entrada              INT IDENTITY(1,1) PRIMARY KEY,
    id_variedad             INT NOT NULL
        CONSTRAINT FK_bitacora_variedad FOREIGN KEY
        REFERENCES variedades(id_variedad),
    tipo_entrada            VARCHAR(50) NULL,
    fecha                   DATE NULL,
    titulo                  NVARCHAR(200) NULL,
    contenido               NVARCHAR(MAX) NULL,
    resultado               VARCHAR(50) NULL,
    id_testblock            INT NULL
        CONSTRAINT FK_bitacora_testblock FOREIGN KEY
        REFERENCES testblocks(id_testblock),
    ubicacion               NVARCHAR(200) NULL,
    usuario                 NVARCHAR(50) NULL,
    fecha_creacion          DATETIME DEFAULT GETDATE()
);
```

Listing 5.42: bitacora_variedades

```
CREATE TABLE asignaciones_testblock (
    id_asignacion           INT IDENTITY(1,1) PRIMARY KEY,
    id_variedad             INT NOT NULL
        CONSTRAINT FK_asigntb_variedad FOREIGN KEY
        REFERENCES variedades(id_variedad),
    id_cuartel              INT NOT NULL
        CONSTRAINT FK_asigntb_cuartel FOREIGN KEY
        REFERENCES cuarteles(id_cuartel),
    cantidad_posiciones     INT NULL,
    fecha_asignacion        DATE NULL,
    estado                  VARCHAR(20) NULL,
    observaciones           NVARCHAR(MAX) NULL,
    fecha_creacion          DATETIME NULL
);
```

Listing 5.43: asignaciones_testblock

5.5 Nivel 4 — Tablas con dependencias de nivel 0–3

```sql
CREATE TABLE movimientos_inventario (
    id_movimiento          INT IDENTITY(1,1) PRIMARY KEY,
    id_inventario          INT NULL
        CONSTRAINT FK_movinv_inventario FOREIGN KEY
        REFERENCES inventario_vivero(id_inventario),
    id_planta              INT NULL
        CONSTRAINT FK_movinv_planta FOREIGN KEY
        REFERENCES plantas(id_planta),
    tipo                   VARCHAR(30) NOT NULL,
    cantidad               INT NOT NULL,
    saldo_anterior         INT NULL,
    saldo_nuevo            INT NULL,
    motivo                 NVARCHAR(200) NULL,
    referencia_destino     VARCHAR(50) NULL,
    usuario                NVARCHAR(50) NULL,
    fecha_movimiento       DATETIME DEFAULT GETDATE()
);
```

Listing 5.44: movimientos_inventario

```sql
CREATE TABLE historial_posicion (
    id_historial           INT IDENTITY(1,1) PRIMARY KEY,
    id_posicion            INT NOT NULL
        CONSTRAINT FK_histpos_posicion FOREIGN KEY
        REFERENCES posiciones_testblock(id_posicion),
    id_planta              INT NULL
        CONSTRAINT FK_histpos_planta FOREIGN KEY
        REFERENCES plantas(id_planta),
    id_planta_anterior     INT NULL
        CONSTRAINT FK_histpos_planta_ant FOREIGN KEY
        REFERENCES plantas(id_planta),
    accion                 VARCHAR(20) NOT NULL,
    estado_anterior        VARCHAR(20) NULL,
    estado_nuevo           VARCHAR(20) NULL,
    motivo                 NVARCHAR(200) NULL,
    usuario                NVARCHAR(50) NULL,
    fecha                  DATETIME DEFAULT GETDATE()
);
```

Listing 5.45: historial_posicion

```sql
CREATE TABLE mediciones_laboratorio (
    id_medicion              INT IDENTITY(1,1) PRIMARY KEY,
    id_posicion              INT NULL
        CONSTRAINT FK_medlab_posicion FOREIGN KEY
        REFERENCES posiciones_testblock(id_posicion),
    id_planta                INT NULL
        CONSTRAINT FK_medlab_planta FOREIGN KEY
        REFERENCES plantas(id_planta),
    temporada                VARCHAR(50) NULL,
```

```sql
    fecha_medicion          DATE NOT NULL,
    fecha_cosecha           DATE NULL,
    brix                    DECIMAL(5,2) NULL,
    acidez                  DECIMAL(5,3) NULL,
    firmeza                 DECIMAL(5,1) NULL,
    calibre                 DECIMAL(5,2) NULL,
    peso                    DECIMAL(6,2) NULL,
    color_pct               INT NULL,
    cracking_pct            INT NULL,
    observaciones           NVARCHAR(MAX) NULL,
    usuario_registro        NVARCHAR(50) NULL,
    fecha_creacion          DATETIME DEFAULT GETDATE(),
    -- Firmeza detallada
    firmeza_punta           DECIMAL(8,2) NULL,
    firmeza_quilla          DECIMAL(8,2) NULL,
    firmeza_hombro          DECIMAL(8,2) NULL,
    firmeza_mejilla_1       DECIMAL(8,2) NULL,
    firmeza_mejilla_2       DECIMAL(8,2) NULL,
    -- Muestra y postcosecha
    n_muestra               INT NULL,
    periodo_almacenaje      INT NULL,
    perimetro               DECIMAL(8,2) NULL,
    pardeamiento            DECIMAL(5,2) NULL,
    traslucidez             DECIMAL(5,2) NULL,
    gelificacion            DECIMAL(5,2) NULL,
    harinosidad             DECIMAL(5,2) NULL,
    color_pulpa             NVARCHAR(50) NULL,
    -- Agronomia
    raleo_frutos            INT NULL,
    rendimiento             DECIMAL(10,2) NULL,
    repeticion              INT NULL,
    -- Color cubrimiento
    color_0_30              INT NULL,
    color_30_50             INT NULL,
    color_50_75             INT NULL,
    color_75_100            INT NULL,
    color_total             INT NULL,
    -- Distribucion color
    color_verde             INT NULL,
    color_crema             INT NULL,
    color_amarillo          INT NULL,
    color_full              INT NULL,
    color_dist_total        INT NULL,
    -- Totales frutos postcosecha
    total_frutos_pardeamiento  INT NULL,
    total_frutos_traslucidez   INT NULL,
    total_frutos_gelificacion  INT NULL,
    total_frutos_harinosidad   INT NULL,
    -- FKs de contexto
```

```
    id_campo                INT NULL
        CONSTRAINT FK_medlab_campo FOREIGN KEY
        REFERENCES campos(id_campo),
    id_variedad             INT NULL
        CONSTRAINT FK_medlab_variedad FOREIGN KEY
        REFERENCES variedades(id_variedad),
    id_especie              INT NULL
        CONSTRAINT FK_medlab_especie FOREIGN KEY
        REFERENCES especies(id_especie),
    id_portainjerto         INT NULL
        CONSTRAINT FK_medlab_portainjerto FOREIGN KEY
        REFERENCES portainjertos(id_portainjerto)
);
```

Listing 5.46: mediciones_laboratorio

```
CREATE TABLE ejecucion_labores (
    id_ejecucion            INT IDENTITY(1,1) PRIMARY KEY,
    id_posicion             INT NULL
        CONSTRAINT FK_ejeclabor_posicion FOREIGN KEY
        REFERENCES posiciones_testblock(id_posicion),
    id_planta               INT NULL
        CONSTRAINT FK_ejeclabor_planta FOREIGN KEY
        REFERENCES plantas(id_planta),
    id_labor                INT NULL
        CONSTRAINT FK_ejeclabor_labor FOREIGN KEY
        REFERENCES tipos_labor(id_labor),
    temporada               VARCHAR(20) NULL,
    fecha_programada        DATE NULL,
    fecha_ejecucion         DATE NULL,
    estado                  VARCHAR(20) DEFAULT 'ejecutada',
    ejecutor                NVARCHAR(100) NULL,
    duracion_min            INT NULL,
    observaciones           NVARCHAR(MAX) NULL,
    usuario_registro        NVARCHAR(50) NULL,
    fecha_creacion          DATETIME DEFAULT GETDATE()
);
```

Listing 5.47: ejecucion_labores

```
CREATE TABLE registros_fenologicos (
    id_registro             INT IDENTITY(1,1) PRIMARY KEY,
    id_posicion             INT NULL
        CONSTRAINT FK_regfenol_posicion FOREIGN KEY
        REFERENCES posiciones_testblock(id_posicion),
    id_planta               INT NULL
        CONSTRAINT FK_regfenol_planta FOREIGN KEY
        REFERENCES plantas(id_planta),
    id_estado_fenol         INT NULL
        CONSTRAINT FK_regfenol_estado FOREIGN KEY
```

```
        REFERENCES estados_fenologicos(id_estado),
    temporada               VARCHAR(20) NULL,
    fecha_registro          DATE NOT NULL,
    porcentaje              INT NULL,
    observaciones           NVARCHAR(MAX) NULL,
    foto_url                NVARCHAR(500) NULL,
    usuario_registro        NVARCHAR(50) NULL,
    fecha_creacion          DATETIME DEFAULT GETDATE()
);
```

Listing 5.48: registros_fenologicos

```
CREATE TABLE alertas (
    id_alerta               INT IDENTITY(1,1) PRIMARY KEY,
    id_posicion             INT NULL
        CONSTRAINT FK_alertas_posicion FOREIGN KEY
        REFERENCES posiciones_testblock(id_posicion),
    tipo_alerta             VARCHAR(50) NULL,
    prioridad               VARCHAR(20) NULL,
    titulo                  NVARCHAR(200) NULL,
    descripcion             NVARCHAR(MAX) NULL,
    valor_detectado         NVARCHAR(50) NULL,
    umbral_violado          NVARCHAR(50) NULL,
    estado                  VARCHAR(20) DEFAULT 'activa',
    usuario_resolucion      NVARCHAR(50) NULL,
    fecha_resolucion        DATETIME NULL,
    notas_resolucion        NVARCHAR(MAX) NULL,
    fecha_creacion          DATETIME DEFAULT GETDATE()
);
```

Listing 5.49: alertas

5.6 Nivel 5 — Tablas finales

```
CREATE TABLE clasificacion_cluster (
    id_clasificacion        INT IDENTITY(1,1) PRIMARY KEY,
    id_medicion             INT NULL
        CONSTRAINT FK_clascluster_medicion FOREIGN KEY
        REFERENCES mediciones_laboratorio(id_medicion),
    cluster                 INT NULL,
    banda_brix              INT NULL,
    banda_firmeza           INT NULL,
    banda_acidez            INT NULL,
    banda_calibre           INT NULL,
    score_total             DECIMAL(5,2) NULL,
    metodo                  VARCHAR(20) NULL,
    fecha_calculo           DATETIME DEFAULT GETDATE()
);
```

Listing 5.50: clasificacion_cluster

```sql
CREATE TABLE umbrales_calidad (
    id_umbral               INT IDENTITY(1,1) PRIMARY KEY,
    id_especie              INT NOT NULL
        CONSTRAINT FK_umbrales_especie FOREIGN KEY
        REFERENCES especies(id_especie),
    metrica                 VARCHAR(30) NOT NULL,
    banda                   INT NOT NULL,
    valor_min               DECIMAL(10,3) NULL,
    valor_max               DECIMAL(10,3) NULL,
    peso_ponderacion        DECIMAL(5,2) NULL,
    activo                  BIT DEFAULT 1,
    fecha_modificacion      DATETIME NULL,
    CONSTRAINT UQ_umbrales UNIQUE (id_especie, metrica, banda)
);
```

Listing 5.51: umbrales_calidad

```sql
CREATE TABLE evidencia_labores (
    id_evidencia            INT IDENTITY(1,1) PRIMARY KEY,
    id_ejecucion            INT NOT NULL
        CONSTRAINT FK_evidencia_ejecucion FOREIGN KEY
        REFERENCES ejecucion_labores(id_ejecucion),
    tipo                    VARCHAR(20) DEFAULT 'foto',
    descripcion             NVARCHAR(200) NULL,
    imagen_base64           NVARCHAR(MAX) NULL,
    url                     NVARCHAR(500) NULL,
    lat                     FLOAT NULL,
    lng                     FLOAT NULL,
    usuario                 NVARCHAR(50) NULL,
    fecha_creacion          DATETIME DEFAULT GETDATE()
);
```

Listing 5.52: evidencia_labores

```sql
CREATE TABLE audit_log (
    id_log                  INT IDENTITY(1,1) PRIMARY KEY,
    tabla                   NVARCHAR(100) NULL,
    registro_id             INT NULL,
    accion                  VARCHAR(20) NULL,
    datos_anteriores        NVARCHAR(MAX) NULL,
    datos_nuevos            NVARCHAR(MAX) NULL,
    usuario                 NVARCHAR(50) NULL,
    ip_address              NVARCHAR(50) NULL,
    fecha                   DATETIME DEFAULT GETDATE()
);
```

Listing 5.53: audit_log

Capítulo 6

Script Python — Migración de Datos

6.1 Configuración (config.py)

```python
"""Configuracion de conexiones origen y destino."""
import os

# ORIGEN: SQL Server Azure (produccion actual)
SOURCE_CONN = (
    "DRIVER={ODBC Driver 18 for SQL Server};"
    "SERVER=tcp:valuedata.database.windows.net,1433;"
    "DATABASE=valuedatadev_2026-01-29T01-40Z;"
    f"UID={os.environ['DB_SOURCE_USER']};"
    f"PWD={os.environ['DB_SOURCE_PASS']};"
    "Encrypt=yes;TrustServerCertificate=no;"
    "Connection Timeout=30;"
)

# DESTINO: SQL Server Azure (adinf)
DEST_CONN = (
    "DRIVER={ODBC Driver 18 for SQL Server};"
    "SERVER=tcp:adinf.database.windows.net,1433;"
    "DATABASE=adinf;"
    "UID=administrador;"
    f"PWD={os.environ['DB_DEST_PASS']};"
    "Encrypt=yes;TrustServerCertificate=no;"
    "Connection Timeout=30;"
)

BATCH_SIZE = 500  # Filas por INSERT batch
```

Listing 6.1: config.py — Connection strings

Peligro

Nunca hardcodear credenciales en el script. Use variables de entorno o Azure Key Vault. El archivo `config.py` solo referencia `os.environ`.

6.2 Orden de tablas para carga

```
# Orden de carga respetando foreign keys
MIGRATION_ORDER = [
    # Nivel 0 - Sin dependencias
    "paises", "regiones", "campos", "especies",
    "portainjertos", "pmg", "origenes", "colores",
    "susceptibilidades", "tipos_labor", "estados_planta",
    "temporadas", "bodegas", "catalogos", "correlativos",
    "marcos_plantacion", "reglas_alerta", "roles", "defectos",
    # Nivel 1 - Dependen de nivel 0
    "comunas", "cuarteles", "pmg_especies",
    "portainjerto_especies", "viveros",
    "estados_fenologicos", "centros_costo",
    "usuarios", "variedades",
    # Nivel 2 - Dependen de nivel 0-1
    "vivero_pmg", "variedad_susceptibilidades",
    "testblocks", "inventario_vivero",
    "detalles_labor", "variedades_log",
    "defectos_variedades",
    # Nivel 3 - Dependen de nivel 0-2
    "testblock_hileras", "posiciones_testblock",
    "plantas", "inventario_testblock",
    "guias_despacho", "paquete_tecnologico",
    "bitacora_variedades", "asignaciones_testblock",
    # Nivel 4 - Dependen de nivel 0-3
    "movimientos_inventario", "historial_posicion",
    "mediciones_laboratorio", "ejecucion_labores",
    "registros_fenologicos", "alertas",
    # Nivel 5 - Dependen de nivel 0-4
    "clasificacion_cluster", "umbrales_calidad",
    "evidencia_labores", "audit_log",
]
```

Listing 6.2: Orden topológico de carga

6.3 Script de migración (`migrate.py`)

```
"""Orquestador de migracion SQL Server -> SQL Server."""
import pyodbc
```

```python
import sys
import time
from config import SOURCE_CONN, DEST_CONN, BATCH_SIZE
from config import MIGRATION_ORDER

# Mapa tabla -> PK para IDENTITY_INSERT
TABLE_PK = {
    "paises": "id_pais", "regiones": "id_region",
    "comunas": "id_comuna", "campos": "id_campo",
    "cuarteles": "id_cuartel", "especies": "id_especie",
    "portainjertos": "id_portainjerto", "pmg": "id_pmg",
    "pmg_especies": "id_pmg_especie",
    "portainjerto_especies": "id_pe",
    "vivero_pmg": "id_vp", "origenes": "id_origen",
    "viveros": "id_vivero", "colores": "id_color",
    "susceptibilidades": "id_suscept",
    "variedades": "id_variedad",
    "variedad_susceptibilidades": "id_vs",
    "tipos_labor": "id_labor",
    "estados_fenologicos": "id_estado",
    "estados_planta": "id_estado",
    "temporadas": "id_temporada",
    "bodegas": "id_bodega",
    "catalogos": "id", "correlativos": "id",
    "centros_costo": "id", "marcos_plantacion": "id",
    "testblocks": "id_testblock",
    "testblock_hileras": "id_hilera",
    "inventario_vivero": "id_inventario",
    "movimientos_inventario": "id_movimiento",
    "posiciones_testblock": "id_posicion",
    "plantas": "id_planta",
    "inventario_testblock": "id_inventario_tb",
    "guias_despacho": "id_guia",
    "historial_posicion": "id_historial",
    "mediciones_laboratorio": "id_medicion",
    "clasificacion_cluster": "id_clasificacion",
    "umbrales_calidad": "id_umbral",
    "registros_fenologicos": "id_registro",
    "detalles_labor": "id_detalle",
    "ejecucion_labores": "id_ejecucion",
    "paquete_tecnologico": "id_paquete",
    "alertas": "id_alerta",
    "reglas_alerta": "id_regla",
    "bitacora_variedades": "id_entrada",
    "usuarios": "id_usuario",
    "roles": "id_rol",
    "audit_log": "id_log",
    "evidencia_labores": "id_evidencia",
    "variedades_log": "id_log",
```

```
    "defectos": "id", "defectos_variedades": "id",
    "asignaciones_testblock": "id_asignacion",
}


def get_columns(cursor, table_name):
    """Obtiene lista de columnas de una tabla."""
    cursor.execute(f"""
        SELECT COLUMN_NAME
        FROM INFORMATION_SCHEMA.COLUMNS
        WHERE TABLE_NAME = ?
        ORDER BY ORDINAL_POSITION
    """, table_name)
    return [row.COLUMN_NAME for row in cursor.fetchall()]


def migrate_table(src_conn, dst_conn, table):
    """Migra una tabla completa con IDENTITY_INSERT."""
    src_cur = src_conn.cursor()
    dst_cur = dst_conn.cursor()

    # Obtener columnas del origen
    columns = get_columns(src_cur, table)
    col_list = ", ".join(f"[{c}]" for c in columns)
    placeholders = ", ".join("?" for _ in columns)

    # Contar registros en origen
    src_cur.execute(f"SELECT COUNT(*) FROM [{table}]")
    total = src_cur.fetchone()[0]

    if total == 0:
        print(f"  [{table}] 0 registros - saltando")
        return 0

    # Activar IDENTITY_INSERT en destino
    dst_cur.execute(
        f"SET IDENTITY_INSERT [{table}] ON"
    )

    # Leer y escribir en batches
    src_cur.execute(
        f"SELECT {col_list} FROM [{table}]"
    )
    migrated = 0
    batch = []

    for row in src_cur:
        batch.append(tuple(row))
        if len(batch) >= BATCH_SIZE:
```

```
            dst_cur.executemany(
                f"INSERT INTO [{table}] ({col_list}) "
                f"VALUES ({placeholders})",
                batch
            )
            migrated += len(batch)
            batch = []
            print(f"  [{table}] {migrated}/{total}", end="\r")

    # Ultimo batch
    if batch:
        dst_cur.executemany(
            f"INSERT INTO [{table}] ({col_list}) "
            f"VALUES ({placeholders})",
            batch
        )
        migrated += len(batch)

    # Desactivar IDENTITY_INSERT
    dst_cur.execute(
        f"SET IDENTITY_INSERT [{table}] OFF"
    )
    dst_conn.commit()

    print(f"  [{table}] {migrated}/{total} OK")
    return migrated


def main():
    print("=" * 60)
    print("MIGRACION SQL Server -> SQL Server")
    print("Garces Fruit - Segmentacion Nuevas Especies")
    print("=" * 60)

    src = pyodbc.connect(SOURCE_CONN)
    dst = pyodbc.connect(DEST_CONN)
    dst.autocommit = False

    results = {}
    errors = []
    t0 = time.time()

    for table in MIGRATION_ORDER:
        try:
            count = migrate_table(src, dst, table)
            results[table] = count
        except Exception as e:
            print(f"  [ERROR] {table}: {e}")
            errors.append((table, str(e)))
```

```
            dst.rollback()

    elapsed = time.time() - t0

    # Resumen
    print("\n" + "=" * 60)
    print("RESUMEN DE MIGRACION")
    print("=" * 60)
    total_rows = sum(results.values())
    print(f"Tablas migradas: {len(results)}/{len(MIGRATION_ORDER)}"
        )
    print(f"Total registros: {total_rows:,}")
    print(f"Tiempo: {elapsed:.1f}s")

    if errors:
        print(f"\nERRORES ({len(errors)}):")
        for t, e in errors:
            print(f"  - {t}: {e}")
        sys.exit(1)

    src.close()
    dst.close()
    print("\nMigracion completada exitosamente.")


if __name__ == "__main__":
    main()
```

Listing 6.3: migrate.py — Orquestador principal

6.4 Script de validación (validate.py)

```
"""Validacion post-migracion: conteos y checksums."""
import pyodbc
from config import SOURCE_CONN, DEST_CONN, MIGRATION_ORDER

def validate():
    src = pyodbc.connect(SOURCE_CONN)
    dst = pyodbc.connect(DEST_CONN)
    src_cur = src.cursor()
    dst_cur = dst.cursor()

    print(f"{'Tabla':<35} {'Origen':>8} {'Destino':>8} {'Estado'}")
    print("-" * 65)

    all_ok = True
    for table in MIGRATION_ORDER:
```

```
        src_cur.execute(f"SELECT COUNT(*) FROM [{table}]")
        src_count = src_cur.fetchone()[0]

        dst_cur.execute(f"SELECT COUNT(*) FROM [{table}]")
        dst_count = dst_cur.fetchone()[0]

        ok = src_count == dst_count
        status = "OK" if ok else "FALLO"
        if not ok:
            all_ok = False

        print(f"{table:<35} {src_count:>8} {dst_count:>8} {status}"
            )

    print("-" * 65)
    if all_ok:
        print("VALIDACION EXITOSA - Todos los conteos coinciden")
    else:
        print("VALIDACION FALLIDA - Revisar tablas con diferencias"
            )

    src.close()
    dst.close()
    return all_ok

if __name__ == "__main__":
    validate()
```

Listing 6.4: validate.py — Validación post-migración

Capítulo 7

Procedimiento Paso a Paso

7.1 Fase 1 — Preparación (Día -1)

1. **Confirmar ventana de mantenimiento** con el equipo de operaciones
2. **Crear la base de datos destino** (vacía) en el nuevo servidor:

```sql
-- BD 'adinf' ya existe en adinf.database.windows.net
-- Verificar que este vacia antes de ejecutar DDL
SELECT COUNT(*) FROM INFORMATION_SCHEMA.TABLES
WHERE TABLE_TYPE = 'BASE TABLE';
```

3. **Instalar dependencias** en la máquina de migración:

```
pip install pyodbc
```

4. **Configurar variables de entorno**:

```
export DB_SOURCE_USER="usuario_origen"
export DB_SOURCE_PASS="clave_origen"
# Destino ya configurado en config.py (adinf.database.windows.
   net)
# Solo se necesita la password como variable de entorno
export DB_DEST_PASS="clave_administrador_adinf"
```

5. **Verificar conectividad** a ambos servidores:

```python
import pyodbc
from config import SOURCE_CONN, DEST_CONN

# Test origen
src = pyodbc.connect(SOURCE_CONN)
print("Origen OK:", src.getinfo(pyodbc.SQL_SERVER_NAME))
src.close()

# Test destino
```

```
dst = pyodbc.connect(DEST_CONN)
print("Destino OK:", dst.getinfo(pyodbc.SQL_SERVER_NAME))
dst.close()
```

6. **Tomar backup** del origen (por seguridad):

```
-- En Azure: Export via Azure Portal o
-- BACPAC export desde SSMS
```

7.2 Fase 2 — Crear estructura (Día 0, hora 0)

1. **Detener la aplicación** (frontend y backend) o ponerla en modo solo lectura
2. **Ejecutar DDL** en la BD destino:

```
# Opcion A: via sqlcmd
sqlcmd -S tcp:adinf.database.windows.net,1433 \
    -d adinf -U administrador -P $DB_DEST_PASS \
    -i 001_create_tables.sql

# Opcion B: via SSMS
# Abrir 001_create_tables.sql y ejecutar (F5)
```

3. **Verificar** que se crearon las 52 tablas:

```
SELECT COUNT(*) AS total_tablas
FROM INFORMATION_SCHEMA.TABLES
WHERE TABLE_TYPE = 'BASE TABLE';
-- Resultado esperado: 52
```

Verificacion

Verificar que el resultado sea exactamente **52 tablas**. Si es menor, revisar el log de ejecución del DDL para identificar errores de sintaxis o dependencias faltantes.

7.3 Fase 3 — Migrar datos (Día 0, hora 0+15min)

1. **Ejecutar el script de migración**:

```
cd backend/scripts/migration/
python migrate.py
```

2. **Monitorear la salida** — cada tabla muestra progreso:

```
============================================================
MIGRACION SQL Server -> SQL Server
Garces Fruit - Segmentacion Nuevas Especies
============================================================
  [paises] 5/5 OK
  [regiones] 16/16 OK
  [campos] 3/3 OK
  [especies] 7/7 OK
  ...
  [plantas] 2291/2291 OK
  [mediciones_laboratorio] 4500/4500 OK
  ...

============================================================
RESUMEN DE MIGRACION
============================================================
Tablas migradas: 52/52
Total registros: 15,234
Tiempo: 45.3s

Migracion completada exitosamente.
```

3. Si hay errores, el script muestra qué tabla falló y por qué. **No continuar** al paso siguiente hasta resolver todos los errores.

Advertencia

El script usa `SET IDENTITY_INSERT ON/OFF` para preservar los IDs originales. Esto es **crítico** para mantener la integridad de las foreign keys. No insertar datos manualmente sin activar esta opción.

7.4 Fase 4 — Validación (Día 0, hora 0+30min)

1. **Ejecutar validación automática**:

```
python validate.py
```

2. **Resultado esperado**:

```
Tabla                                  Origen  Destino Estado
-----------------------------------------------------------------------

paises                                      5        5 OK
regiones                                   16       16 OK
campos                                      3        3 OK
...
plantas                                  2291     2291 OK
```

```
mediciones_laboratorio                4500      4500 OK
...
----------------------------------------------------------------------

VALIDACION EXITOSA - Todos los conteos coinciden
```

3. **Validaciones manuales adicionales**:

```sql
-- Verificar integridad referencial
SELECT 'plantas sin posicion valida' AS check_name,
    COUNT(*) AS errores
FROM plantas p
LEFT JOIN posiciones_testblock pt
    ON p.id_posicion = pt.id_posicion
WHERE p.id_posicion IS NOT NULL
    AND pt.id_posicion IS NULL

UNION ALL

SELECT 'mediciones sin planta valida',
    COUNT(*)
FROM mediciones_laboratorio m
LEFT JOIN plantas p ON m.id_planta = p.id_planta
WHERE m.id_planta IS NOT NULL
    AND p.id_planta IS NULL

UNION ALL

SELECT 'variedades sin especie valida',
    COUNT(*)
FROM variedades v
LEFT JOIN especies e ON v.id_especie = e.id_especie
WHERE e.id_especie IS NULL;
-- Resultado esperado: 0 errores en cada fila
```

4. **Reseed IDENTITY** para que nuevos INSERTs usen el ID correcto:

```sql
-- Para cada tabla, resetear el IDENTITY al maximo ID
   existente
DECLARE @max_id INT;

SELECT @max_id = ISNULL(MAX(id_planta), 0)
FROM plantas;
DBCC CHECKIDENT('plantas', RESEED, @max_id);

-- Repetir para cada tabla (o automatizar con cursor)
```

Verificacion

La migración es exitosa cuando:

- `validate.py` reporta **todos OK**
- Las queries de integridad referencial retornan **0 errores**
- Los IDENTITY seeds están correctamente posicionados

7.5 Fase 5 — Cutover (Día 0, hora 0+45min)

1. **Actualizar connection string** en el backend (`.env` o Azure App Configuration):

```
DATABASE_URL="mssql+pyodbc://administrador:$DB_DEST_PASS
@tcp:adinf.database.windows.net,1433/adinf
?driver=ODBC+Driver+18+for+SQL+Server&Encrypt=yes"
```

2. **Reiniciar el backend** para que tome la nueva conexión
3. **Verificar funcionalidad básica**:
   - Login de usuario admin
   - Listar testblocks
   - Ver detalle de una planta con mediciones
   - Crear un registro de prueba (ej: medición de laboratorio)
4. **Monitorear logs** del backend por 15 minutos buscando errores de conexión o integridad

7.6 Fase 6 — Rollback (solo si falla)

Si la validación o el cutover fallan:

1. **Revertir connection string** al origen
2. **Reiniciar backend** apuntando a la BD original
3. **Documentar** el error encontrado
4. **No borrar** la BD destino — se usa para diagnóstico
5. **Programar** nueva ventana de migración con el fix aplicado

Capítulo 8

Consideraciones Especiales

8.1 Columnas VARBINARY (imágenes)

Las tablas `portainjertos`, `variedades` y `defectos` contienen columnas `VARBINARY(MAX)` con imágenes codificadas. El script `migrate.py` las maneja nativamente como bytes de Python, sin necesidad de conversión.

Advertencia

Las imágenes pueden ser grandes (varios MB). Si la migración es lenta en estas tablas, considere aumentar el `Connection Timeout` en `config.py` y reducir `BATCH_SIZE` a 50.

8.2 Tabla `usuarios` y hashes bcrypt

Los hashes bcrypt en `password_hash` se migran tal cual — son strings de 64 caracteres. Los usuarios podrán hacer login con sus mismas credenciales en el destino sin necesidad de resetear contraseñas.

8.3 Tabla `audit_log`

Se migra el histórico completo. Considere si desea:

- Migrar todo (recomendado para trazabilidad)
- Migrar solo los últimos N meses
- Archivar en tabla separada (`audit_log_historico`)

8.4 Collation

El origen usa `SQL_Latin1_General_CP1_CI_AS`. Asegúrese de que el destino tenga la **misma collation** para evitar problemas con acentos, eñes y comparaciones de texto:

```sql
-- Verificar collation del destino
SELECT DATABASEPROPERTYEX(DB_NAME(), 'Collation');
-- Debe retornar: SQL_Latin1_General_CP1_CI_AS
```

8.5 Reseed automático de IDENTITY

Después de cargar datos con `IDENTITY_INSERT ON`, el seed del IDENTITY no se actualiza automáticamente. Se debe ejecutar `DBCC CHECKIDENT` para cada tabla. El siguiente script lo automatiza:

```sql
DECLARE @sql NVARCHAR(MAX) = '';

SELECT @sql = @sql +
    'DECLARE @max_' + TABLE_NAME + ' INT; ' +
    'SELECT @max_' + TABLE_NAME +
    ' = ISNULL(MAX(' +
    COLUMN_NAME + '), 0) FROM [' +
    TABLE_NAME + ']; ' +
    'DBCC CHECKIDENT(''' + TABLE_NAME +
    ''', RESEED, @max_' + TABLE_NAME + '); '
FROM INFORMATION_SCHEMA.COLUMNS
WHERE COLUMNPROPERTY(
    OBJECT_ID(TABLE_SCHEMA + '.' + TABLE_NAME),
    COLUMN_NAME, 'IsIdentity') = 1;

EXEC sp_executesql @sql;
PRINT 'IDENTITY reseed completado para todas las tablas.';
```

Listing 8.1: Reseed automático de todas las tablas

Capítulo 9

Checklist de Migración

Cuadro 9.1: Checklist completo de migración

Paso	Check	Descripción
Preparación		
1		Ventana de mantenimiento confirmada
2		BD destino creada con collation correcta
3		Python + pyodbc instalados
4		Variables de entorno configuradas
5		Conectividad verificada a ambos servidores
6		Backup del origen realizado
Ejecución		
7		Aplicación detenida / modo solo lectura
8		DDL ejecutado — 52 tablas creadas
9		`migrate.py` ejecutado sin errores
10		`validate.py` reporta todos OK
11		Queries de integridad referencial OK
12		IDENTITY reseed ejecutado
Cutover		
13		Connection string actualizado al destino
14		Backend reiniciado
15		Login de admin funciona
16		CRUD básico verificado
17		Monitoreo de logs OK por 15 min
Post-migración		
18		Documentar resultado final
19		Notificar al equipo
20		Archivar scripts y logs de migración

Capítulo 10

Apéndice A — Variables de Entorno

Cuadro 10.1: Variables de entorno requeridas

Variable	Descripción
`DB_SOURCE_USER`	Usuario con permisos de lectura en BD origen (valuedata)
`DB_SOURCE_PASS`	Contraseña del usuario origen
`DB_DEST_PASS`	Contraseña del usuario `administrador` en BD destino (adinf)

Informacion

El servidor destino (`adinf.database.windows.net`), la base de datos (`adinf`) y el usuario (`administrador`) están fijos en `config.py`. Solo la contraseña se parametriza como variable de entorno por seguridad.

Capítulo 11

Apéndice B — Troubleshooting

11.1 Errores comunes

Cuadro 11.1: Errores frecuentes y solución

Error	Solución
`IDENTITY_INSERT is already ON for table X`	Solo una tabla puede tener IDENTITY_INSERT ON a la vez. Si el script anterior falló, ejecutar `SET IDENTITY_INSERT [X] OFF` manualmente.
`Cannot insert duplicate key`	La tabla destino ya tiene datos. Limpiar con `DELETE FROM [tabla]` antes de reintentar, o usar `TRUNCATE TABLE [tabla]` si no tiene FKs.
`Foreign key constraint violation`	Las tablas se están cargando en orden incorrecto. Verificar que `MIGRATION_ORDER` respeta las dependencias.
`Login timeout expired`	Verificar firewall rules en Azure. Agregar la IP de la máquina de migración en "Networking" del SQL Server.
`String data right truncation`	Una columna en el destino tiene tamaño menor al dato. Comparar DDL origen vs destino para esa columna específica.
`pyodbc.InterfaceError: ODBC Driver not found`	Instalar ODBC Driver 18: `apt-get install msodbcsql18` (Linux) o descargarlo de Microsoft (Windows).

11.2 Cómo reintentar una tabla específica

Si una tabla falla, puede migrarla individualmente:

```
import pyodbc
from config import SOURCE_CONN, DEST_CONN
from migrate import migrate_table

src = pyodbc.connect(SOURCE_CONN)
dst = pyodbc.connect(DEST_CONN)
dst.autocommit = False

# Limpiar tabla destino primero
dst.cursor().execute("DELETE FROM [nombre_tabla]")
dst.commit()

# Reintentar migracion
migrate_table(src, dst, "nombre_tabla")

src.close()
dst.close()
```

Capítulo 12

Apéndice C — Historial de Versiones

Cuadro 12.1: Historial de versiones del documento

Versión	**Fecha**	**Autor**	**Cambios**
1.0	2026-04-10	Equipo Técnico	Versión inicial